LeakyLinks: Measuring the Security and Privacy Risks of URL Scanning Services (Artifacts)

LeakyLinks: Measuring the Security and Privacy Risks of URL Scanning Services (Artifacts) - Ali Mustafa, Jannis Rautenstrauch, Florian Hantke, Shubham Agarwal, Stefano Calzavara, Ben Stock, GitHub.

- Published: date not stated
- Original: <https://github.com/cispa/leakylinks>
- Preserved from: https://github.com/cispa/leakylinks (git) on 2026-08-19
- Repository commit: a94de83622865368dd30915eeface824fdc003b0
- Licence: see the repository

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/cispa/leakylinks>
- Commit: `a94de83622865368dd30915eeface824fdc003b0`
- Documents preserved: 2

LICENSE

Blob `2817751d44cb`, 1068 bytes, at commit `a94de8362286`.

MIT License

Copyright (c) 2026 Ali Mustafa

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

LeakyLinks

This repository contains the open-source artifact of:

"**LeakyLinks: Measuring the Security and Privacy Risks of URL Scanning Services**"

Accepted at IEEE Symposium on Security and Privacy (S&P) 2026.

Overview

The LeakyLinks framework identifies SPI URLs by analyzing data from multiple URL scanning services. It processes URLs through a multi-stage pipeline:

1. **Scraping**: Collects URLs from 6 URL scanning services
2. **Live Crawl**: Visits URLs and captures before/after snapshots to detect session state
3. **Token Detection**: Identifies high-entropy tokens in URLs (potential session identifiers)
4. **Page Difference Check**: For URLs without tokens, compares before/after pages to detect session state changes
5. **Screenshot Analysis**: Analyzes screenshots of potentially sensitive URLs using vision-based LLM

The 6 URL Scanning Services

- Anyrun
- Cloudflare Radar
- Hybrid-Analysis
- Joe Sandbox
- URLQuery
- URLScan

Pipeline Architecture

Data Flow

1. **Scraper** (`scraper/`): Continuously collects URLs from the 6 services and stores them in service-specific result tables (`*_results`)
1. **Database Triggers**: Automatically create entries in `analysis_output` table when new URLs are scraped
1. **Pipeline Workers** (run in sequence):
   - **CrawlWorker** (`--crawl`): Visits each URL twice (before/after dropping session) and captures snapshots
   - **URLTokenCheckWorker** (`--url_token_check`): Detects high-entropy tokens in the final URL
   - **PageDifferenceCheckWorker** (`--page_difference_check`): Only processes URLs without tokens; compares before/after pages to detect session state
   - **ScreenshotAnalysisWorker** (`--spi_detector`): Analyzes screenshots for URLs that have tokens OR page differences

Pipeline Phases

The pipeline uses `task_phase_status` table to track progress through phases:

- **live_crawl**: Visit URL, capture before/after snapshots, store in `live_crawl_analysis` JSON
- **url_token_check**: Check if `finalUrlBefore` contains high-entropy tokens sets `finalurlbefore_has_token`
- **page_difference_check**: Only for URLs with `finalurlbefore_has_token = False` ; compares HTML similarity sets `page_different`
- **spi_detector**: Only for URLs with `(finalurlbefore_has_token = True OR page_different = True)` ; analyzes screenshots for sensitive content

Key Concepts

- **State Drop**: The process of visiting a URL twice - once normally, then again after dropping session cookies/values. If the page content differs, it indicates the URL is an SPI URL. This is implemented in the `live_crawl` phase and analyzed in the `page_difference_check` phase.
- **analysis_output table**: Central table that tracks all URLs through the pipeline. Contains:
  - `live_crawl_analysis` : JSON with before/after snapshots and redirects
  - `finalurlbefore_has_token` : Boolean flag set by token detection
  - `page_different` : Boolean flag set by page difference check
  - `has_redirection` : Boolean flag indicating redirects occurred

Quickstart (With docker and docker compose installed)

1. Build and start the services

```
docker compose up -d --build
```

1. Exec into the main application container

```
docker compose exec leakylinks bash
```

1. Add fake scraped examples to the database

```
python config/fake_plugin_fill.py examples
```

1. Run the pipeline phases in order:

```
# Phase 1: Live crawl (visits URLs, captures snapshots)
python pipeline/pipeline/run_pipeline.py --crawl

# Phase 2: Token detection (checks for high-entropy tokens in URLs)
python pipeline/pipeline/run_pipeline.py --url_token_check

# Phase 3: Page difference check (only for URLs without tokens)
python pipeline/pipeline/run_pipeline.py --page_difference_check

# Phase 4: Screenshot analysis (for URLs with tokens or page differences)
python pipeline/pipeline/run_pipeline.py --spi_detector
```

Components

- **Scraper** (`scraper/`): Collects data from the 6 URL scanning services. It gathers details like the URL, screenshot URL, and results from the API. Runs continuously to accumulate data over time.
- **URL Token Checker** (`url_token_checker/`): Parses URLs (with full path+query), applies basic checks, then uses entropy analysis to detect high-entropy tokens and flag potentially sensitive URLs.
- **Live Crawl** (`live_crawl/`): Visits URLs twice (with and without session values) to capture before/after snapshots. This implements the "State Drop" technique to detect SPI URLs.
- **Page Difference Checker** (`page_difference_checker/`): Compares before/after HTML pages to detect session state changes. Only processes URLs that don't have tokens.
- **Screenshot Analyzer** (`spi_detector/`): Processes screenshots from URLs that have tokens or showed page differences, using vision-based LLM analysis to detect sensitive content. Performs concurrent batch processing with checkpointing support.
- **Honey** (`honey/`): Infrastructure for the honeypot experiment including submitters and the base honeypage used.

Configuration

- The pipeline configuration is located in `config/settings.py`

- Use `.env` as a reference for environment variables
- The model used in the actual project was `qwen3-vl:30b-a3b-instruct-q8_0` which needs more than 34 GB of VRAM, but this docker uses `qwen3-vl:2b-instruct` to make it smaller. The docker compose will only finalize when the LLM is downloaded and ready. Make sure to have 8 GB of VRAM.

Database Schema

The main tables are:

- `*_results`: Service-specific tables storing scraped URLs
- `analysis_output`: Central table tracking URLs through the pipeline
- `task_phase_status`: Tracks progress through pipeline phases
- `screenshot_analysis_results`: Stores screenshot analysis results

Contact

Ali Mustafa — ali.mustafa@cispa.de

Citation

The paper will be available at the IEEE Computer Society Digital Library after publication.

```
@INPROCEEDINGS {,
author = { Mustafa, Ali and Rautenstrauch, Jannis and Hantke, Florian and Agarwal, Shubham and Calzavara, Stefano a
booktitle = { 2026 IEEE Symposium on Security and Privacy (SP) },
title = {{ LeakyLinks: Measuring the Security and Privacy Risks of URL Scanning Services }},
year = {2026},
volume = {},
ISSN = {2375-1207},
pages = {834-853},
abstract = { URL scanning services are widely used in security workflows to detect malicious websites and protect users
keywords = {},
doi = {10.1109/SP63933.2026.00130},
url = {https://doi.ieeecomputersociety.org/10.1109/SP63933.2026.00130},
publisher = {IEEE Computer Society},
address = {Los Alamitos, CA, USA},
month =May}
```